THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

TEXT CLASSIFICATION WITH MACHINE LEARNING

Predicting News Article Topics Using Linear Models, RNNs, and Transformers

By: Jonathan Beck

AG™ NEWS

Business Problem & Objectives

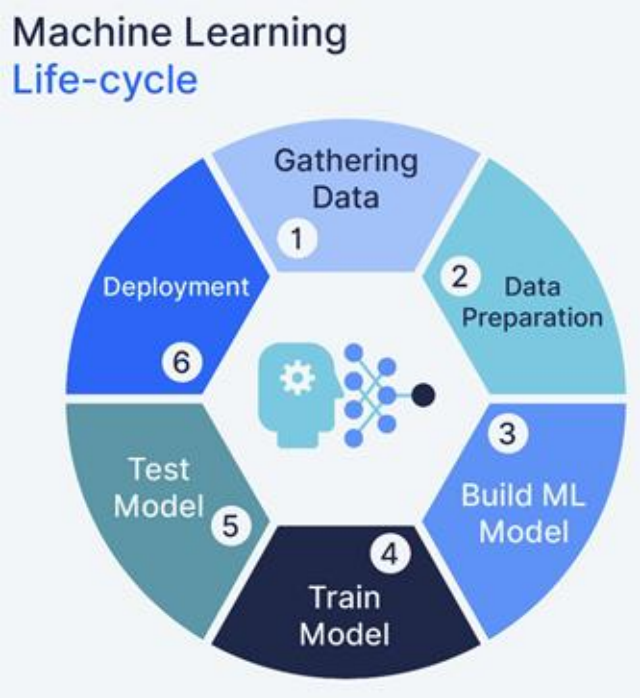
Why This Problem Matters

- Organizations ingest massive volumes of text every day
- Manual labeling is slow, inconsistent, and expensive
- Automated topic classification improves:
  - Content recommendation
  - News aggregation
  - Trend monitoring
  - User experience

Core Question: *"What modeling approach provides the optimal balance of accuracy, cost, and complexity for accurately classifying news articles?"*

Project Goals

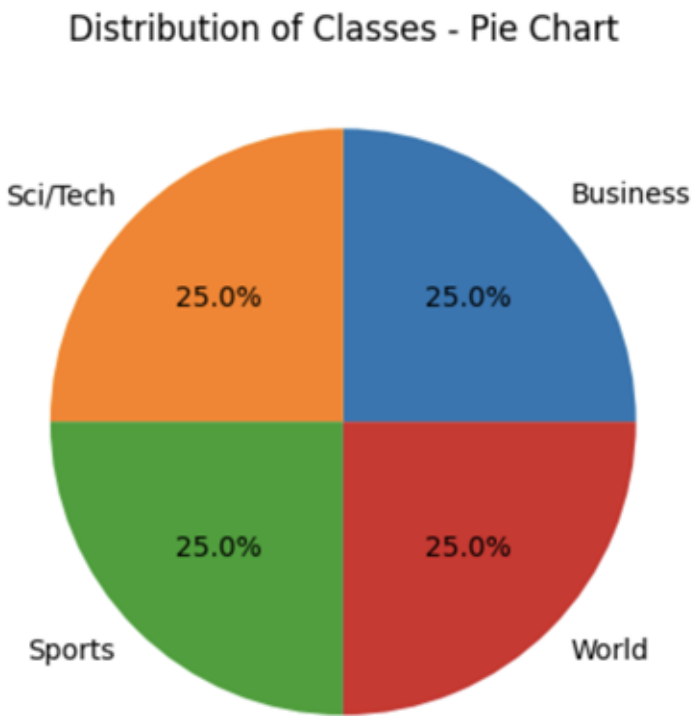
1. Build a clean and auditable preprocessing pipeline
   a. Standardized text fields
   b. Controlled feature variations (title-only vs description-only vs both)
2. Establish defensible baseline performance
   a. Naive Bayes
   b. Regularized Logistic Regression
3. Evaluate advanced architectures
   a. LSTM-based RNN
   b. RoBERTa Transformer (transfer learning)
4. Compare accuracy, runtime, and practicality
   a. Identify the "best" model for *real-world deployment*, not just accuracy.



Data Overview

AG News Dataset (Kaggle)

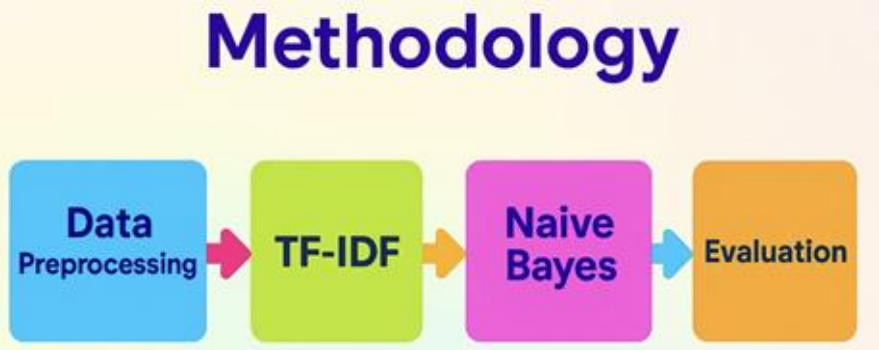
- 120,000 training articles
- 7,600 test articles
- Perfectly balances across four classes:
  - World, Sports, Business, Sci/Tech
- Key Properties
  - No missing values
  - Structured text fields
    - Titles = short, sparse signals
    - Descriptions = richer, more informative



Preprocessing & Design: Baseline Models

Naive Bayes (TF-IDF)

- Text normalization
- Unigram + bigram tokenization
- Stop-word removal
- Vocabulary capped at 30K, ranked by term frequency across corpus
  - determines number of columns in feature matrix (regularization control)
- TF-IDF Weighting
  - Higher score implies token is important to document
  - Downweights tokens found frequently across corpus
- L2 normalization
  - ensures model not biased by document length
- Sparse matrix
  - Shape (n_documents, 30K)



Preprocessing & Design: Baseline Models

Logistic Regression (TF-IDF)

- Similar text preprocessing as Naive Bayes
- Two stage 5-Fold Stratified Cross Validation Grid Search to speed up training and fine-tuning
- First Stage: Fine-tune logistic regression parameters
  - Unigrams + top 20K features
  - Solver set to lbfgs (well-suited for high dimensional spaces), L2 (ridge) penalty term and a search through four regularization, C, terms to control how heavily the model penalizes large weights on the features
- Second Stage: Apply logistic parameters to determine optimal TFIDF column length and n-gram range

Preprocessing & Design: Deep Learning

RNN (LSTM)

- All terms across corpus tokenized and normalized, token frequencies counted and ranked to assign IDs to each token
  - Top 30000 kept (number of embeddings learned across entire corpus - regularization)
- Sequence tokenization at embedding layer
  - 200 tokens per input sequence (time steps per article)
  - Shorter articles padded with zeros, longer articles truncated
- LSTM (128 units)
  - Compresses all text embeddings into single vector of length 128 to be fed into classification head

Transformer (RoBERTa)

- Subword tokenization
  - Smaller segments of words mapped to pretrained RoBERTa vocabulary, which effectively enables model to handle any word
- Max-length: 128 tokens (article token input length, provided tokes IDs)
- Start and end tokens applied (boundary signals).
  - Enables use of first token embedding as summary vector (contextualized embedding for every token in article sequence)
- Frozen pretrained backbone
- 768 neurons in dense feedforward network
  - Weights updated with a light learning rate (3e-5)
- Multinomial classification head (softmax)
- Early stopping (validation loss monitored, restore best weights if loss doesn't improve after two epochs

Baseline Model Results

Naive Bayes

- Fast, interpretable, strong baseline
- Best accuracy with title + description
- Alpha = 0.1 performed best (light smoothing)

```
preds = []

for a in alphas:
  nb = MultinomialNB(alpha=a)
  nb.fit(X_train_vec, y_train)
  preds.append(nb.predict(X_val_vec))
```

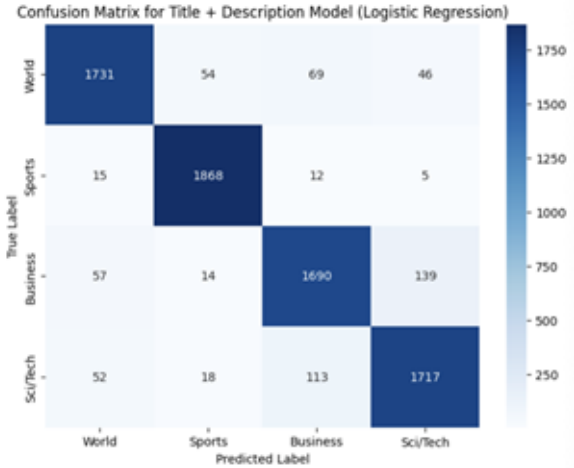
```
# Laplace Smoothing = 0.1
classificationSummary( y_val, preds[0], class_names=class_names)
```

```
Confusion Matrix (Accuracy 0.9126)

         Prediction
  Actual    World   Sports Business Sci/Tech
   World     2691      108      116       85
  Sports       21     2957       12       10
Business       94       18     2607      281
Sci/Tech      110       16      178     2696
```

Logistic Regression

- Outperformed Naive Bayes
  - Accuracy of ~ 92% on holdout test set
- Best configuration: {C=1, L2 penalty, 40K vocabulary, Unigrams + bigrams}
  - Moderate shinkage of weights, prevents any weights being driven to zero entirely



Critical Insight: Combining text + description fields as predictors outperformed single-field models across the board.

RNN (LSTM) Results

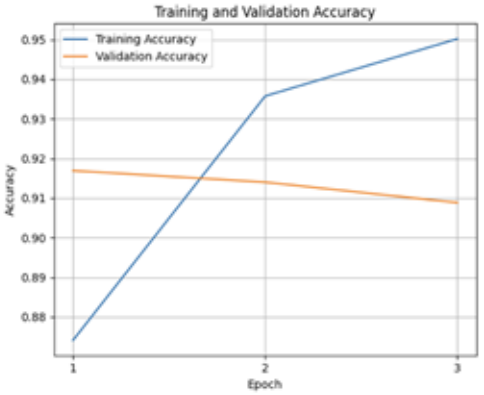
Why Use an RNN?

- Captures sequential structure
  - reads sequence of article token inputs step-by-step
- Learns contextual relationships
  - Single learned embedding representation of whole sequence fed into classification head
- Disadvantages
  - Struggle with long range dependencies

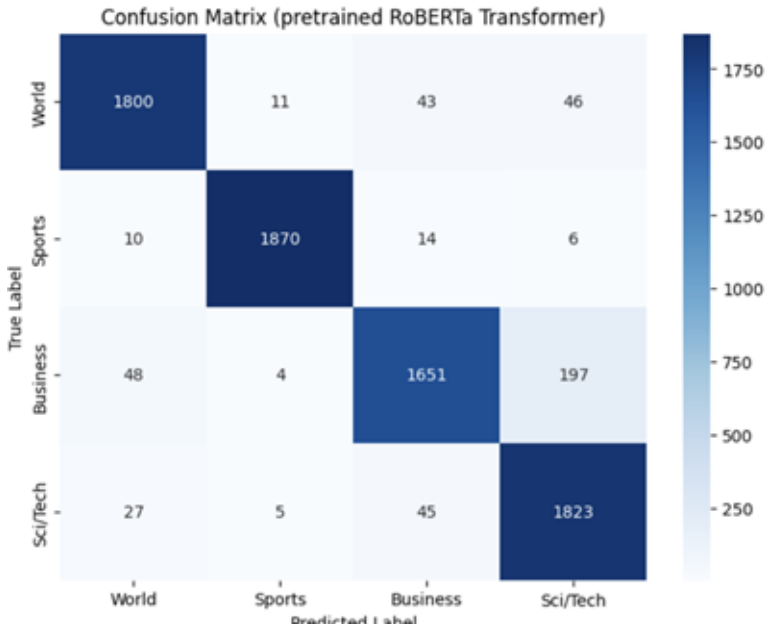
Performance

- Validation accuracy plateaued after 3 epochs
- Similar to Logistic Regression in terms of accuracy
  - Significantly higher training cost

Takeaway: RNNs add complexity without meaningful performance uplift for this classification task.



Transformer (RoBERTa) Results



Why RoBERTa?

- Contextual embeddings
- Learns long-range dependencies
- Transfer learning: strong performance with limited training

Training Strategy

- Frozen backbone + fine-tuned classification head
- Early stopping after 3 epochs
- Avoided full fine-tuning due to training cost: 45-minutes per epoch

Results

- 94% accuracy on unseen, test data
- Balanced across all classes
- Best overall performance.

Cross-Model Insights

Text Field Choice Matters

- Title alone: limited signal
- Descriptions: richer vocabulary, stronger boundaries
  - less overlap between class clusters
- Combined fields: best performance across all models

Complexity vs Value

- Logistic Regression
  - Fast, interpretable, high accuracy
- RNNs
  - More complex, higher computational cost for no real gain
- Transformers
  - Best accuracy, but highest computational cost

Practicality Depends on Use Case

- **High-stakes classification = Transformer; Fast and efficient deployment = Logistic Regression**

Business-Oriented Recommendations

Recommended Baseline: TF-IDF + Logistic Regression architecture

- High accuracy
- Transparent decision rules
- Minimal infrastructure requirements
- Trains in seconds

When to Consider Transformers

- Premium content recommendations
- High-precision classification tasks
- Scenarios where marginal accuracy gains justify cost

Strategic Guidance

- **Start simple & scale complexity only when business value demands it**

Conclusion

- Clean preprocessing pipelines matter
- Linear models remain competitive
- Deep learning is powerful but computationally expensive
- Modeling selection is a *strategic* decision, not just a technical one

FINAL TAKEAWAY: A well-tuned Logistic Regression model offers the best balance of accuracy, interpretability, and efficiency, while Transformers provide the premium selection when cost is no barrier